Robotics Course Project Report: Robot Build, Odometry and Control

Mohamed Islem, TH Köln, Robotics course (Team 3)

Assumptions

- Parts 1 and 2 are summarised from your earlier project notes, so they have less detail than the odometry lab.
- Distances are measured with a ruler from the start line. Odometry values come from the Arduino serial printout.
- "Forward" means the right wheel turns CW and the left wheel CCW. "Backward" is the opposite.
- The last tuning round is not confirmed. The final values are in the sketch, and the latest straight-line and turn results still need a confirmation run (see Current state and next steps).

Project overview

The course project is a two-wheeled robot that drives and turns accurately on its own. It is built in weekly steps, and most of the time went into odometry and control (Part 3). The robot now drives 200 cm lines within a few cm on good runs and runs the square routine. The latest settings still need one confirmation run.

Part	Weekly project	Due	Points	What was done	Status
1	Intro VS Code, Python, GitHub	22.10.	0.5	VS Code, Python and Git set up on the Raspberry Pi and PC	Working, sketch pushed to GitHub
2	Ubuntu, ROS2, Arduino, servos	29.10.	1.5	Wiring, ROS2 serial link, first servo calibration, first drive tests	Working
3	Odometry and control	12.11.	9	Feedback angles, P-controller, odometry, covariance matrix, 200 cm test, square routine	Final tuning to confirm

The sketch marks the square routine as Project 5 in its comments, so it may count for a later weekly project.

Part 1: Tools, VS Code, Python and GitHub

The code is written and uploaded from VS Code on the Raspberry Pi, and versioned with Git on GitHub. You started this part new to VS Code and Git, so the commands were done step by step.

- **Repository:** `https://github.com/lboo3/Advanced-Robotics-TH.git`, working branch `Laboratory_Into_ROS2_team3`.
- **Workflow:** change the sketch on the Pi, then `git add`, `git commit`, `git push`. On the PC, `git clone -b Laboratory_Into_ROS2_team3 <url>` once, then `git pull`.

- **Problems met:** the commands pasted as one block, and the harmless error "remote origin already exists". Details are in the odometry lab part, under Git and GitHub.

Part 2: Wiring, ROS2 serial link and first servo calibration

The robot drove from the first tests, but it drifted 20 to 30 cm over 1 m (forward to the right, backward to the left). The fix was a proper calibration, then the P-controller in Part 3.

Item	Value
Servo control pins	left D3, right D5
Feedback pins	left D2, right D4
Stop signal	1500 µs on both servos, wheels stayed still
Feedback check	hand-turning a wheel was measured on both sides
First `write()` calibration	stop 94, CW 91, CCW 97 (92 and 96 moved but stuttered)
Base speeds first used	CW 88, CCW 100, stop 94
Encoder calibration, wheels in the air	47 µs (left) and 51 µs (right) from stop gave 146 and 144 deg/s
Wheel diameter	65 mm
Wheel base	10 cm in the early notes, 9.38 cm in the first sketch, 9.9 cm in the final sketch

ROS2 link: the Pi runs the node `serial_node` / `serial_subscriber` from `~/team3_ws`. Commands (`f`, `b`, `s`) are sent with `ros2 topic pub` as `std_msgs/String` on `/teamthree_serial_ACM0`. One `f` command lasted about 200 ms, so 6 commands moved the robot about 1 m, and it ended about 40 cm to the right.

Challenge: speeds set in the code reset on every restart, and the measurement noise is about ±5 deg/s (1 µs is about 5% speed). **Solution:** the calibrated values are written into the sketch, and the 90° turns were first checked with the wheel-distance method (7.07 turned correctly, 7.85 turned about 10° too far).

The wheel base in the final sketch (9.9 cm) is the one the latest odometry numbers in Part 3 use. Earlier numbers used 9.38 cm.

Part 3: Odometry and control, summary

The goal was a robot that drives straight for 200 cm forward and 200 cm back, and turns accurately for the autonomous square routine. We worked through a compile error, tool problems on the Raspberry Pi, servo resolution, turn overshoot, and mainly a straight-line drift. A calibration sweep showed that the two servos behave differently by direction. We added per-wheel, per-direction corrections and tuned them with ruler measurements. The drift dropped from about 15 to 20 cm to a few cm on good runs. The final sketch also slows down before each stop, counts the coasting after every stop, and sets the

controller factors per direction. The latest settings still need one confirmation run. The final sketch was pushed to GitHub.

Part 3: Setup

- **Robot:** two Parallax Feedback 360° High-Speed Servos, wheel diameter 6.5 cm (circumference about 20.4 cm), wheelbase 9.9 cm in the final sketch.
- **Controller:** Arduino Nano 33 BLE, programmed from a Raspberry Pi with `arduino-cli` (`arduino:mbed_nano:nano33ble`, port `/dev/ttyACM0`).
- **Feedback:** each servo gives a PWM position signal (about 910 Hz, duty cycle 2.9% to 97.1%). The sketch converts it to an angle, tracks wrap-arounds, and accumulates `totalTheta` per wheel.
- **Control:** a proportional controller compares the distance covered by the two wheels and speeds up the wheel that lags.
- **Odometry:** x, y and heading are computed from the wheel increments.
- **Test commands:** `g` (200 cm forward then 200 cm back), `q` (square routine), `k` (calibration sweep), `x` (reset pose).

Part 3: Challenges and solutions

Compile error: stray `r`

- **Problem:** the upload failed with `'r' was not declared` and `'i' was not declared`.
- **Cause:** a lone `r` on line 175 where `case 'r':` was meant. The second error was a follow-on from the first.
- **Solution:** restore `case 'r':` (or delete the stray line). A curly quote in a `case` line causes the same kind of error.

Drift to the right: servo command resolution

- **Problem:** the robot drifted about 5 cm to the right over the forward leg, and about 20 cm off the start point after forward plus back.
- **Cause:** `servo.write()` takes whole numbers, and one step is about 10 µs, which is a large jump near cruising speed. A correction of 1 step was too coarse.
- **Solution:** send the controller correction in microseconds with `writeMicroseconds()`, and raise the gain from 6 to 10, then to 30 with a cap of 60 µs.
- **Result:** forward Y odometry improved from -5.30 cm to -1.89 cm. The wheels were then within about 0.4 cm of each other on each leg.

Is the sensor wrong? Ruled out

- **Question:** could a false extra turn be added near the 357° to 3° dead zone of the feedback signal? The 20 cm offset is close to one wheel circumference (20.4 cm).
- **Check:** `distRight` and `distLeft` were both about 200 cm on each leg, and odometry Y matched the measured drift.
- **Conclusion:** the sensors and odometry are trustworthy. The drift is a physical effect that equal wheel angles do not capture.

Scale factor `kRight` and random scatter

- **Problem:** after the gain change, the backward leg still ended about 15 cm to the right. The controller kept the wheels at equal angles, so the cause had to be unequal ground distance per rotation.
- **Attempts:** a single factor `kRight = 1.007` on the right wheel. It made things worse because I chose the wrong direction. The next value was 0.993.
- **Insight:** the robot did not give the same result each run, so no single fixed factor could be "exact". It contains a systematic part (correctable) and a random part (slip, battery, load) that cannot be removed by a constant.
- **Suggestion not adopted:** a heading-based correction in the controller. We chose to fix the servo calibration first.
- **Outcome:** `kRight` was removed.

Turn overshoot and the failed slow-down

- **Problem:** in the square routine the robot "jumped" when it stopped, and each turn overshot by about 0.7° or more. Because the coasting after the stop was not counted, the odometry heading drifted.
- **Attempt:** slow both wheels for the last 25° of a turn.
- **Why it failed:** the slow speeds (92 and 96) fell inside the servo's zero-speed deadband (1480 to 1520 µs, ±15), so the robot literally stopped before reaching the target angle. The slow-down was removed. This was my mistake.
- **Solution kept:** after the stop and the 300 ms delay, read the feedback once more and update odometry, so the coasting is counted in the pose.
- **Alternative:** stop slightly early (`while (fabs(actualAngle) < targetAngle - 1.0)`) and tune the margin from the `Angle error` lines.

Update: the slow-down came back in a safer form. The final `turn()` runs at full speed until 15° before the target, then drops to write values 90 and 98 (4 steps from stop). Those sit outside the deadband and still move the wheels, and the square routine log shows no stalls. It stops 3° before the target and counts 300 ms of coasting.

Calibration sweep: the root cause

A new command `k` was added. It runs both wheels from write value 84 to 104, settles for 1.5 s, and measures for 3 s per value, printing degrees per second. The wheels were lifted.

Value	Right (deg/s)	Left (deg/s)
86 (CW)	-332	-346
93 to 95	about 0	about 0
102 (CCW)	+321	+306
103 (CCW)	+347	+335

Findings:

1. The stop point (about 93 to 95) is correct for both wheels, so `speedStop = 94` is not the problem.
2. The left wheel is about 12% slower on CCW (306) than on CW (346). The right wheel is close (321 vs 332). One shared CW/CCW pair cannot suit both wheels. That explains why the error depends on direction.
3. The speed changes by about 45 deg/s per write step, so whole-number writes are too coarse for matching wheels.

Per-wheel, per-direction trims

- **Solution:** each wheel gets its own trim in microseconds for CW and CCW (`trimRightCW`, `trimRightCCW`, `trimLeftCW`, `trimLeftCCW`), applied through `usRight()` and `usLeft()`. The starting values were 3, 0, 6, 3 µs, taken from the sweep.
- **Why it is better:** the baseline speeds are already close to equal, so the P-controller only handles the small remainder.
- **Test method:** run `g` five times, press `x` first, and judge by averages, because single runs scatter.

Direction-dependent controller factor `kR`

After the trims, five runs gave the following (ruler):

Leg	Result
Forward (200 cm mark)	4, 5, 7, 9, 8 cm to the right
Back at the start	first run perfect, the others about 3 cm off

After lowering `trimRightCW` to 0, forward ended about 10 cm to the right at the same spot on four runs, and backward was about 2 cm. Since the P-controller forces equal wheel angles, it overrides small trim changes. So a factor was added inside the controller, chosen by direction:

```
double kR = (baseSpeedR < speedStop) ? forwardValue : backwardValue;
double err = (fabs(totalThetaRight) * kR - fabs(totalThetaLeft)) * wheelPerimeter / 360.0;
```

The first number applies only to the forward leg, the second only to the backward leg. `turn()` does not use this controller.

The tuning history:

Forward / backward value	Forward (ruler)	Backward (ruler)
0.995 / 1.0	about on the line (1 cm left)	about 20 cm right
0.995 / 1.010	1 to 10 cm left	about 30 cm right (worse)
0.997 / 0.980	not reported	shifted to the left
0.997 / 0.990	not reported	not reported yet

Two lessons here. First, I got the direction of the backward factor wrong at 1.010, which made the curve worse. The data showed the backward value must be below 1.0. Second, one log showed the backward

wheels equal within 0.3 cm, which meant the new value was not on the board. A failed compile leaves the old code running, so always confirm the upload finished without errors.

The last suggestion, for a target of about 2 cm right forward and 5 cm right backward, was `0.9965` forward and `0.985` backward. It has not been tested yet.

Values in the latest sketch: `kR` is 0.996 forward and 0.994 backward. The trims are 0 for the right wheel (both directions), 6 for the left wheel CW and 3 for the left wheel CCW.

Tool problems on the Raspberry Pi

- **Problem:** `arduino-cli` was not found. After `sudo snap install arduino-cli`, running it gave `Exec format error`.
- **Cause:** the snap binary did not match the CPU architecture of the Pi.
- **Solution:** remove the snap, check whether the earlier `arduino-cli` is still in `~/bin` or `/usr/local/bin`, or reinstall with the official install script, then make sure the `arduino:mbed_nano` core is installed. Use `uname -m` to see the architecture if the install fails.

Git and GitHub

- **Goal:** push the sketch from the Pi to GitHub and download it on the PC.
- **Steps:** `git init` in `~/team3_ws`, remote `origin` set to `https://github.com/lboo3/Advanced-Robotics-TH.git`. The error "remote origin already exists" was harmless, because an earlier pasted block had already added it.
- **Branch:** the target was the existing branch `Laboratory_Into_ROS2_team3` (4 commits ahead of `main`). The Pi repo was linked to it with `git fetch origin` and `git checkout -b Laboratory_Into_ROS2_team3 origin/Laboratory_Into_ROS2_team3`, then the sketch was committed and pushed.
- **Login:** GitHub needs a personal access token instead of the account password.
- **On the PC:** `git clone -b Laboratory_Into_ROS2_team3 https://github.com/lboo3/Advanced-Robotics-TH.git`, then `git pull` for later changes.

Stopping smoothly: deceleration in `move()`

- **Problem:** a hard stop made the robot lurch, and the stopping error carried into the next turn.
- **Solution:** in the last 12 cm (half the distance on moves under 24 cm) the speed ramps down to 30% of cruise. Odometry keeps running for 300 ms after the stop, so coasting is counted.
- **Result:** the measured distance matched the target within 0.04 cm in the log (40.00 for 40, 56.59 for 56.57, 56.61 for 56.57).

Turn accuracy

- **Measurement:** over 14 turns (6 single turns and the 8 turns of the square routine) the odometry angle overshot the target by 1.47° on average, between 0.58° and 2.64°, always in the same direction.
- **Cause:** the loop stops 3° early, but the wheels coast about 4.5° after the stop.
- **Fix:** set `stopAngle = targetAngle - 4.5`. The spread of about ±0.6° is the limit of this method.

- **Wheel base:** the real angle is only right if `wheelBase` is right. It was changed from 9.38 to 9.9 cm. To correct it, compare one printed turn angle with a protractor: `wheelBase_new = wheelBase * odo / real`.
- **Straight moves in the square:** the first six `move()` calls curved left by 3° to 6° each, and the last two went straight. This is why the final pose of the square (X 11 cm, Y -12 cm) is not closed.

Serial menu not printing at start

- **Cause:** the Nano 33 BLE talks over native USB, so text printed in `setup()` is lost before the monitor connects. A second cause is that `calculateThetaRight()` waits forever if no feedback signal arrives.
- **Solution:** `while (!Serial && millis() < 3000);` after `Serial.begin(9600);` (added to the sketch). If the menu is still missing, check servo power and the feedback wires.

Accidental push of build folders to GitHub

- **Problem:** a broad `git add` pushed `build`, `install`, `log`, `src`, `.vscode` and a nested repo folder to the branch.
- **Solution:** remove them from Git only, without deleting the files: `git rm -r --cached <folder>`, then commit and push. From now on, add only the sketch file: `git add arduino/team3_servosketch/team3_servosketch.ino`.

Part 3: Results overview

Stage	Forward	Backward
Start (write(), gain 6)	about 5 cm right	about 20 cm off start
µs corrections, gain 30	about 2 cm (odometry)	about 15 cm right (ruler)
Per-wheel trims	4 to 9 cm right	0 to 3 cm off start
`kR` 0.995 forward	about on the line	about 20 cm right

The latest sketch uses kR 0.996 forward and 0.994 backward, plus smooth stopping. Ruler results for these exact values were not reported yet, so they are the next measurement.

Part 3: Lessons learned

1. Match the control resolution to the problem. Whole-number `write()` steps are too coarse for fine corrections, so use microseconds.
2. Trust the data before changing the model. The sensor check showed the sensors were fine and the error was physical.
3. Calibrate each wheel in each direction. The sweep found the real cause, a 12% CCW/CW speed gap on the left wheel.
4. A constant cannot remove random error. Judge tuning by averages over several runs.
5. Slowing near the target is unsafe if the speed falls into the servo deadband.
6. Check the direction of every correction with one measurement before scaling it.
7. Confirm each upload finished without errors before judging a test.

Part 4 A : Straight Line Covariance

To evaluate the reliability and the repeatability of how the robot moves, 30 forward tests were conducted manually, in which the robot was commanded to travel a forward distance of 40 cm. Figure 1 shows how these measurements were conducted.

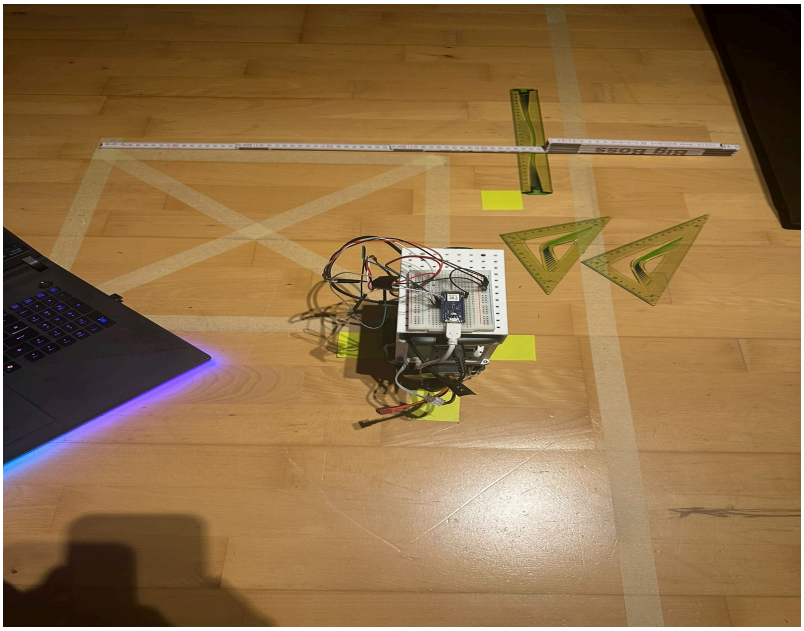
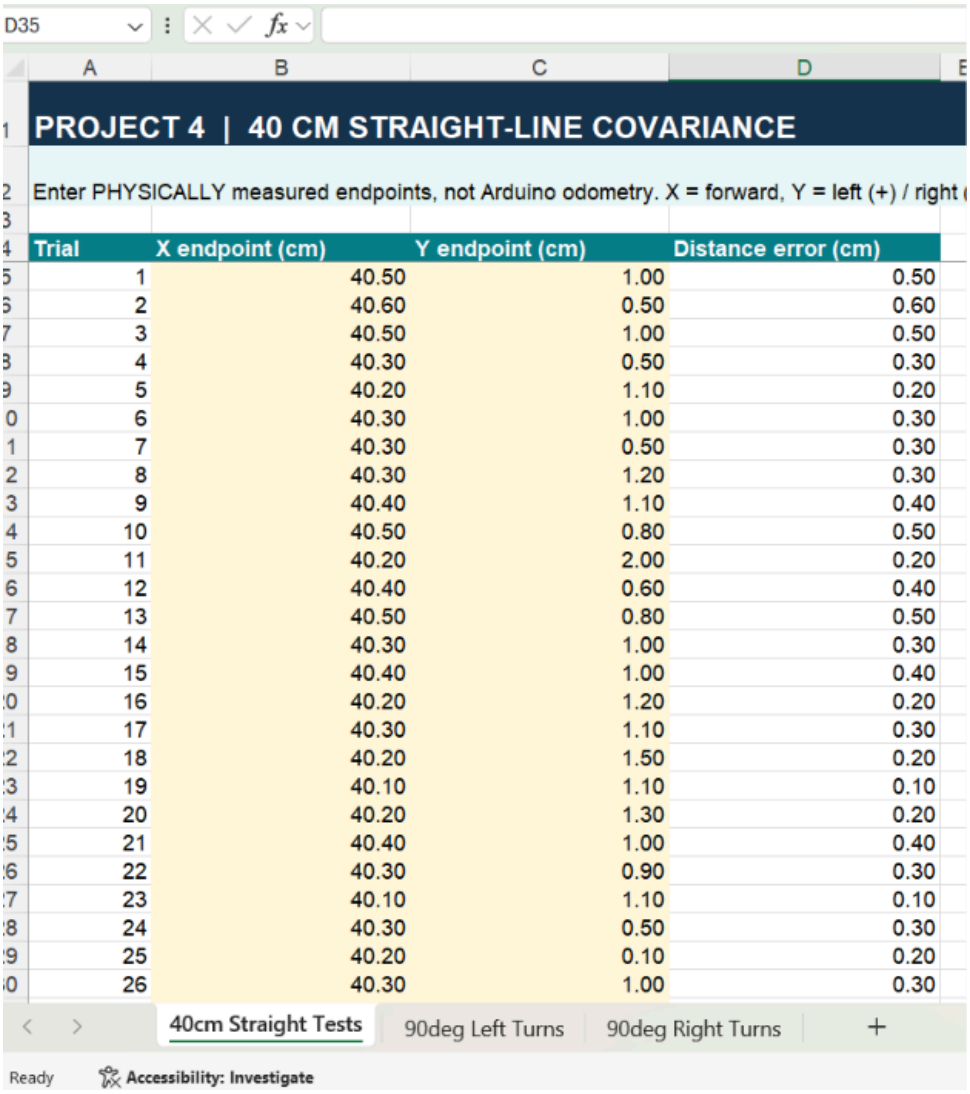


Figure 1 Covariance/Variance Manual Measurement devices

Figure 2 shows the 30 trial data points The robot was placed in the exact same starting position and orientation before each trial run. The final position was then measured in both the X and Y directions, where X and Y represent forward and lateral displacements respectively. The average final position was 40.317 cm. Compared to the target position of (40, 0) cm, the robot travelled an average of 0.317 cm beyond the desired commanded distance while also having lateral displacement of 0.953 cm in the positive CCW direction. These results show that there are bits of systematic errors in both the forward travel and lateral positioning.



PROJECT 4 | 40 CM STRAIGHT-LINE COVARIANCE

Enter PHYSICALLY measured endpoints, not Arduino odometry. X = forward, Y = left (+) / right

Trial	X endpoint (cm)	Y endpoint (cm)	Distance error (cm)
1	40.50	1.00	0.50
2	40.60	0.50	0.60
3	40.50	1.00	0.50
4	40.30	0.50	0.30
5	40.20	1.10	0.20
6	40.30	1.00	0.30
7	40.30	0.50	0.30
8	40.30	1.20	0.30
9	40.40	1.10	0.40
10	40.50	0.80	0.50
11	40.20	2.00	0.20
12	40.40	0.60	0.40
13	40.50	0.80	0.50
14	40.30	1.00	0.30
15	40.40	1.00	0.40
16	40.20	1.20	0.20
17	40.30	1.10	0.30
18	40.20	1.50	0.20
19	40.10	1.10	0.10
20	40.20	1.30	0.20
21	40.40	1.00	0.40
22	40.30	0.90	0.30
23	40.10	1.10	0.10
24	40.30	0.50	0.30
25	40.20	0.10	0.20
26	40.30	1.00	0.30

40cm Straight Tests | 90deg Left Turns | 90deg Right Turns

Ready | Accessibility: Investigate

Figure 2 Data Points For Straight-Line Covariance

The covariance matrix was calculated using the 30 measured endpoints as seen below in Figure 3.

RESULT	VALUE
Completed pairs	30.0000
Mean X (cm)	40.3167
Mean Y (cm)	0.9533
Pxx (cm²)	0.0147
Pxy = Pyx (cm²)	-0.0136
Pyy (cm²)	0.1405
Std dev X (cm)	0.1213
Std dev Y (cm)	0.3748

COVARIANCE MATRIX	X	Y
X	0.0147	-0.0136
Y	-0.0136	0.1405

Target endpoint: (40 cm, 0 cm). Matrix uses 1/N (population covariance).

Figure 3 Straight Line Covariance Matrix Results

The diagonal elements of the matrix represent the variance in the X and Y directions. The variance in the forward direction was 0.01472 cm², whereas the lateral variance was 0.14049 cm². The information displays the robot's final position varied more in the Y direction than in the X.

The corresponding standard deviations were 0.121 cm in X and 0.375 cm in Y. The lateral standard deviation was approximately three times greater than the forward standard deviation, indicating that the robot was more consistent in the distance travelled than in maintaining a straight path. This was verified also by the eye test as the robot seemed to constantly stop within 1 cm of the 40cm target whereas with straight forward movement, depending on the trial the robot would drift.

The off-diagonal elements represent the covariance between the X and Y measurements. The value of −0.01356 cm² indicates a weak negative relationship between forward and lateral displacement. In these measurements, trials with a greater forward displacement tended to have a slightly smaller lateral displacement.

The results suggest that the robot can reproduce the commanded travel distance with relatively little variation, but maintaining its heading during straight-line movement is less consistent. Small differences in wheel speeds, servo response or wheel-ground interaction could contribute to the observed lateral variation.

It is also important to distinguish between accuracy and repeatability. The covariance matrix describes how much the final positions vary between trials, but it does not directly measure the error relative to the commanded target. Although the forward variance was low, the mean endpoint still showed a small forward overshoot and lateral offset.

Part 4 B : Turning Variance

To evaluate the repeatability of the robot's rotational movement, 30 left turns and 30 right turns were performed, each with a commanded rotation of 90°. The actual turning angles were measured and used to calculate the mean angle and variance for each direction.

Both turning directions produced mean angles slightly below the commanded 90°, indicating a small tendency to under rotate and not reach the desired turn angle. The average angular error was 0.867° for left turns and 0.700° for right turns.

The turning variances were similar, with the right turns showing a slightly higher variance. The standard deviations of approximately 1.18° indicate that the measured turning angles varied by a similar amount in both directions.

Overall, the measurements indicated that the robot has relatively consistent forward movement and comparable turning repeatability in both directions. However, the greater lateral variation during straight-line movement suggests that heading errors have a noticeable effect on the final position. Improving wheel-speed balancing and heading control could help reduce this variation in future tests.

Figures 4 and 5 show the variance in the 90 degree left and right turns respectively.

PROJECT 4 | 90DEG LEFT TURNS

Enter actual measured final angles in degrees; reposition robot to identical start before each turn.

Trial	Measured angle (°)	Error vs 90° (°)
1	89.00	-1.00
2	89.00	-1.00
3	89.00	-1.00
4	89.00	-1.00
5	91.00	1.00
6	90.00	0.00
7	87.00	-3.00
8	88.00	-2.00
9	88.00	-2.00
10	91.00	1.00
11	89.00	-1.00
12	87.00	-3.00
13	89.00	-1.00
14	90.00	0.00
15	91.00	1.00
16	88.00	-2.00
17	91.00	1.00
18	89.00	-1.00
19	90.00	0.00
20	89.00	-1.00
21	88.00	-2.00
22	88.00	-2.00
23	90.00	0.00
24	90.00	0.00
25	87.00	-3.00
26	90.00	0.00

RESULT	VALUE
Completed trials	30
Mean angle (°)	89.1333
Variance (deg²)	1.3822
Std deviation (°)	1.1757
Mean absolute error (°)	1.1333

Figure 4 90 Degree Left Turn Variance

PROJECT 4 | 90DEG RIGHT TURNS

Enter actual measured final angles in degrees; reposition robot to identical start before each turn.

Trial	Measured angle (°)	Error vs 90° (°)
1	89.00	-1.00
2	88.00	-2.00
3	91.00	1.00
4	88.00	-2.00
5	88.00	-2.00
6	88.00	-2.00
7	89.00	-1.00
8	89.00	-1.00
9	92.00	2.00
10	88.00	-2.00
11	91.00	1.00
12	91.00	1.00
13	89.00	-1.00
14	89.00	-1.00
15	91.00	1.00
16	89.00	-1.00
17	88.00	-2.00
18	89.00	-1.00
19	89.00	-1.00
20	90.00	0.00
21	90.00	0.00
22	89.00	-1.00
23	89.00	-1.00
24	88.00	-2.00
25	91.00	1.00
26	88.00	-2.00

RESULT	VALUE
Completed trials	30
Mean angle (°)	89.3000
Variance (deg²)	1.4100
Std deviation (°)	1.1874
Mean absolute error (°)	1.2333

Figure 5 90 Degree Right Turn Variance

Current state and next steps

- **Done:** per-wheel trims, direction-dependent `kR` (0.996 forward, 0.994 backward), smooth stopping in `move()`, adaptive turns with coasting counted, the `k` calibration command, the serial menu fix, and the sketch on GitHub (branch `Laboratory_Into_ROS2_team3`).
- **To confirm:** run `g` five times (press `x` first) and measure each leg from the line. Adjust `kR` by about 2 cm per 0.001 forward and about 1 cm per 0.001 backward.
- **Turns:** set `stopAngle = targetAngle - 4.5`, then check one `r` turn with a protractor and correct `wheelBase` if the real angle differs.
- **Git:** push only the sketch file, and remove the accidentally pushed folders with `git rm -r --cached`.
- **Optional later:** a heading-based correction to reduce the random part of the drift.